

Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Introduction to Python
- 2 Programming in Python
- 3 Scalar Objects
 - Type `int()`
 - Type `float()`
 - Type `bool`
 - Type Conversions
- 4 Expressions
 - Arithmetic Operations on `int` and `float`: a Recap
 - Comparison Between `ints` and `floats`: a Recap
 - Operator Precedence
- 5 References

Introduction to Python

Why Python?

- A **high-level** language for **beginners**
- with a **clean syntax**, but...
- **Industrial strength** programming language used at thousands of companies (one of three official Google languages)
- Free, well documented, well supported

Recall

We will use Python 3 (there are non-backwards compatible differences with respect to Python 2). Further, Python 2 is now unsupported.

A (Python) program is composed by a sequence of:

- **definitions** (also **expressions**) which are **evaluated**
- **statements** (also **commands**) which are **executed**

In plain terms...

- **expressions** yield some results (e.g., $5 * 5$)
- **statements** instruct the interpreter to do something (e.g., print on screen, assign to variable)

Expressions and statements can either

- be directly written into a shell (**interactive mode**)
- be written into a text file and then loaded into the shell (**script mode**).

Interactive vs. Script Mode

Interactive Mode: perfectly fine for testing and learning.

Type your commands/expressions into the shell and Python will execute/evaluate those immediately:

```
>>> 5 * 5    ←input  
25           ←output
```

Script Mode: is what you normally do.

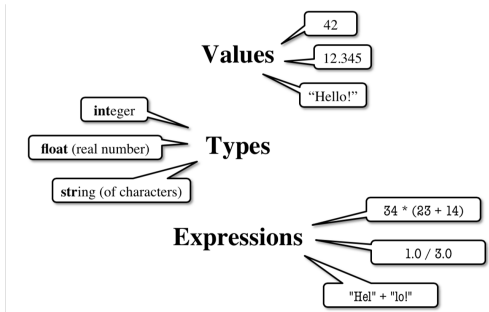
- write the entire source code in a text file ending with the `.py` extension (say `myscript.py`)
- `myscript.py` will be executed in its entirety by the Python interpreter

Programming in Python

- programs manipulate **data objects**
- each data object has its own **type**, which defines the type of operations that can be done with them
- objects can either be:

Scalar: cannot be subdivided

Non-scalar: have an inner structure that can be accessed



Scalar Objects

- **int**: represents integers (e.g., 5)
- **float**: represents real numbers (e.g., 3.14)
- **bool**: represents Boolean values (**True** or **False**)
- **NoneType**: special with only one value (**None**)

Hint

Use the `type()` function if you are not sure about the data type.

Examples:

```
>>> type(109)    ←input  
<class 'int'>    ←output
```

```
>>> type(10.9)   ←input  
<class 'float'> ←output
```

Values:

$\dots, -3, -2, -1, 0, 1, 2, 3, \dots$

Integer literals look like a sequence of numbers with no periods nor commas.

Operations:

- $+$, $-$, $*$, $/$ [plain arithmetic]
- $**$ [to the power of]
- unary $-$ [negative numbers]
- $\%$ [modulo, remainder]

Note

In Python 3, operations between integers may not yield an integer.

Examples:

```
>>> 7/5    ←input
1.4        ←output
>>> 7//5    ← // is the floor division!
1
```


Values:

some (approximated) range of $\mathbb{R}$

Note that:

- numbers with a "." are decimals (e.g., 2.0)
- numbers with no decimal separator are integers (e.g., 2)

Operations:

- +, -, *, / [plain arithmetic]
- ** [to the power of]
- unary - [negative numbers]
- % [modulo, remainder]

Note

In Python 3, operations between floats or between a float and an integer, always yield a float.

Indeed:

```
>>> 7.0/5.0    ←input
1.4            ←output
>>> 7.0//5      ← // is the floor division!
1.0
```

Python stores floating point numbers as base 2 (binary) fractions:

$$\{\text{number}\} = \{\text{integer mantissa}\} \cdot 2^{-\{\text{exponent}\}}$$

For example: $0.125 = 1 \cdot 2^{-3} = 1 \cdot \frac{1}{2^3} = 1 \cdot \frac{1}{8}$.

Warning

Not all numbers can be represented in base 2 fractions. Python chooses the best approximation it can. The same also holds for some numbers in base 10 fraction. What about $1/3$?

Python may yield some **approximation errors**, that propagate as expressions go through.

Example:

```
>>> 0.1+0.2
```

←input

```
0.30000000000000004
```

←output

```
>>> 0.1+0.2+0.1+0.2
```

```
0.6000000000000001
```

```
>>> 0.1+0.2+0.1+0.2+0.1+0.2+0.1+0.2
```

```
1.2000000000000002
```

Values:

True and **False** (capitalized!)

Operations:

- $\text{not } a = \begin{cases} \text{True} & \text{if } a \text{ is False} \\ \text{False} & \text{if } a \text{ is True} \end{cases}$ [NOT operator]
- $a \text{ and } b = \begin{cases} \text{True} & \text{if both } b \text{ and } a \text{ are True} \\ \text{False} & \text{otherwise} \end{cases}$ [AND operator]
- $a \text{ or } b = \begin{cases} \text{True} & \text{if } a \text{ is True or } b \text{ is True} \\ \text{False} & \text{otherwise} \end{cases}$ [OR operator]

Booleans are the output of comparisons between items (e.g., **int** or **float**):

Order: $i < j$, $i > j$, $i \leq j$, $i \geq j$

(In)equality: $i \neq j$, $i == j$

Table 1: Truth Table between two Boolean variables a and b

a	b	not a	not b	a and b	a or b
True	True	False	False	True	True
False	False	True	True	False	False
True	False	False	True	False	True
False	True	True	False	False	True

Switching between `float` and `int` can be performed via **casting** (**explicit** conversion):

- `float(2)` converts value 2 (integer) to 2.0 (floating point)
- `int(2.9)` converts value 2.9 (floating point) to 2 (integer)

Warning

`int()` does not round to the nearest integer: rather, it truncates the decimal part. To round to the nearest integer, use `round()` instead.

Python supports **widening** automatically (**implicit** conversion):

$\text{bool} \rightarrow \text{int} \rightarrow \text{float}$
narrow to wide

Widening: Python does it automatically, if needed

Example: `1/0.5=2.0` because Python casts 1 to `float`

Narrowing: Python never does it automatically

Because narrowing leads to information loss!

Expressions

- **Expressions** are combinations of **objects** and **operators**
- an expression yields a **value**, which has its own **type**
- the simplest expression has the form:

`<object, operator, object>`

Table 2: Recap of the common arithmetic operations between two numerical variables i and j

Syntax	Operator	Output Type
$i+j$	Sum	if both of them are floats, result is float
$i-j$	Difference	if both of them are int, result is int
$i*j$	Product	if either of them is float, result is float
i/j	Division	result is float (always)
$i\%j$	Remainder (Modulo)	if both of them are floats, result is float
$i**j$	Power of	if both of them are int, result is int
$i//j$	Floor division	if either of them is float, result is float

Note

Comparisons below evaluate to a `bool`.

<code>i < j</code>	→	returns <code>True</code> if <code>i</code> is smaller than <code>j</code>
<code>i <= j</code>	→	returns <code>True</code> if <code>i</code> is smaller than or equal to <code>j</code>
<code>i > j</code>	→	returns <code>True</code> if <code>i</code> is greater than <code>j</code>
<code>i >= j</code>	→	returns <code>True</code> if <code>i</code> is greater than or equal to <code>j</code>
<code>i == j</code>	→	returns <code>True</code> if <code>i</code> is equal to <code>j</code>
<code>i != j</code>	→	returns <code>True</code> if <code>i</code> is not equal to <code>j</code>

Comparisons can be chained arbitrarily and comparisons have the same priority:

`i < j < k` is the same as `i < j` and `j < k`

Since comparisons do not act in a pairwise fashion, ‘weird’ comparisons like

`i < j > k`

are perfectly valid.

Precedence between operators can be ‘forced’ via parenthesis.

In principle, operators follow the **PEMDAS** order:

- ① **P**arenthesis
- ② **E**xponential
- ③ **M**ultiplication and **D**ivision
- ④ **A**ddition and **S**ubtraction

The Big Picture

- 1 Parenthesis: (...)
- 2 Exponentiation: **
- 3 Unary operators: + and -
- 4 Binary arithmetic: *, /, // and %
- 5 Binary arithmetic: + and -
- 6 Comparisons: >, <, >=, <=, ==, !=
- 7 Logical not
- 8 Logical and
- 9 Logical or

Note

Same line means same precedence. Read ties left to right.

Hint

In case of emergency, use parenthesis!

References

- Gutttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Section 2.1]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 1.2–1.5, 2.5, 5.1–5.3]
- <https://docs.python.org/3/tutorial/floatingpoint.html>
- <https://www.geeksforgeeks.org/chaining-comparison-operators-python/>